AS AYANDA SHONGWE · BCOM ACCOUNTING

A COMPLETE WALKTHROUGH

Four projects, every concept, and how to explain them

South African payroll, tax and SARS compliance software — and the reasoning behind every line of it. Written so that you can sit down with an interviewer, a lecturer or a client and talk about it with confidence.

PROJECTS	TESTS	LINES OF CODE	DEPENDENCIES
4	262	11.5k	0

Ayanda Shongwe

21 October 2026

What is in here

The first half explains what was built and the ideas behind it. The second half is the part to rehearse: the questions you are likely to be asked, and a way to answer each one.

01 The one-minute version p.3
02 What was built p.4
03 The accounting you need p.7
04 How the code fits together p.11
05 The concepts, one by one p.14
06 The design system p.20
07 Testing p.22
08 How it is hosted p.23
09 The mistakes I made p.24
10 Questions, with answers p.26
11 The numbers p.31

> **How to use this document.** Read Part 1 to 4 once, properly. Then use Part 10 as a script. Every answer in it is written the way I would actually say it out loud — short, specific, and with the number in it. An interviewer trusts a candidate who can say "R56 172, because 44 118 plus 26% of the amount over R245 100, less the R17 820 primary rebate" rather than one who says "I used the SARS tables".

What is in each part

PART	WHAT IT GIVES YOU
01 — The one-minute version	What to say when someone asks "so what did you build?"
02 — What was built	The four projects, with a diagram of how they relate
03 — The accounting	The tax rules themselves, worked through, because this is what you know that a computer science student does not
04 — Architecture	How each project is put together, and why
05 — The concepts	Twenty-two named concepts: what they are, where they live, why they matter, and one sentence for each
06 — The design system	Colour, type, space and motion, with the reasons
07 — Testing	What is tested, what is not, and why the hand-worked numbers matter
08 — Hosting	How it runs for free, and the trade-offs

PART	WHAT IT GIVES YOU
09 — The mistakes	The bugs I found and fixed. This is the best material in the document.
10 — Questions	Twenty-two questions with model answers

01 The one-minute version

If you only memorise one paragraph, memorise this one.

> **"I build payroll and tax software for South African businesses, and every number in it comes from the SARS tables rather than from what I thought the rule was."**

That sentence works because it says three things at once: the **domain** (payroll and tax in South Africa), the **differentiator** (the numbers are right, and I can prove it), and the **attitude** (I check the source instead of guessing).

Then, if they want more, there are three directions you can go, and you should pick the one that fits the person you are talking to:

To a recruiter

Lead with the **payroll suite**. It is the biggest thing, it has a database, an API and a dashboard, and it solves a problem a real business has. Mention that it has 148 tests and that the tax figures are checked against SARS's own published tables.

To an accountant

Lead with the **domain knowledge**. The annualised method, the s11F cap, the year-end reconciliation, the EMP201 against the IRP5. This is where a BCom student has an unfair advantage and should use it.

To a developer

Lead with the **engineering**. Zero dependencies, the engine separated from the database, a JSON API with no framework, and the same algorithm in Python and JavaScript with shared tests.

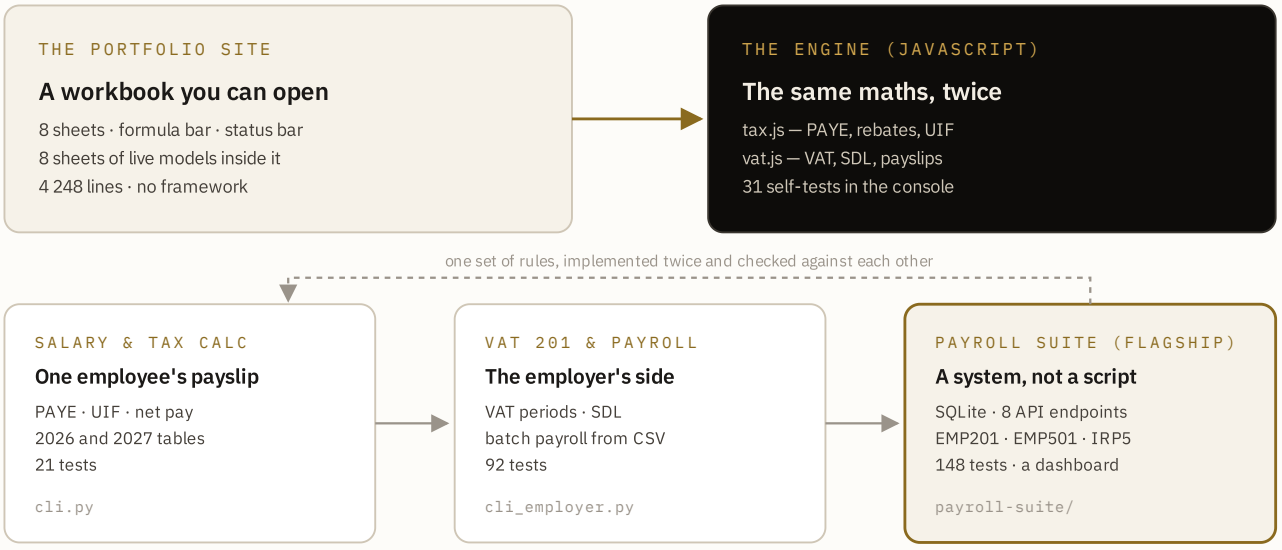
The four things, in one line each

PROJECT	WHAT IT DOES	WHY IT EXISTS
Payroll Suite Python · SQLite · API · dashboard	A payroll system: employees in a database, monthly payslips, EMP201 declarations, IRP5 certificates, a deadline calendar and a dashboard.	It is the flagship. It shows the whole employer half of tax, and it is the one an employer could actually use.
VAT 201 & Payroll Suite Python · CLI · CSV	Output tax less input tax, the six bi-monthly VAT periods with their real due dates, SDL, and a batch payroll run from a CSV.	It is the middle rung: the employer's side of tax, in the terminal, with a batch mode.
Salary & Tax Calculator Python · CLI	What one employee pays: PAYE, UIF, the net pay, on the official 2026 and 2027 tables.	It is where it started, and it is the project that proves the numbers.
The portfolio site	A workbook you can open: eight sheets, a formula bar that shows how every number was worked out, and	It is the shop window, and it is the only piece a visitor can touch.

HTML · CSS · JavaScript	the live models inside it.	

02 What was built

Four projects, built in order. Each one needed the one before it, so they read as a single body of work rather than four separate assignments.



How the four projects relate. The browser engine and the Python engine implement the same rules independently, and the same test cases run against both — that is what keeps them honest.

Why there are four, and not one

Each project exists because the previous one ran into a wall.

- **The calculator** asked "what does one person pay?" It needed the SARS tables, and it needed them to be right — so it needed tests with hand-worked answers.
- **The employer tool** asked "what does a business owe?" That brought in VAT, SDL, the EMP201 and a batch of employees, which a single payslip calculator could not hold.
- **The payroll suite** asked "what happens when it is a real business?" That needed a database, history, forms that reconcile and a dashboard somebody could actually look at.
- **The site** asked "how does anyone find out?" It needed the models to run in a browser, which meant porting the engine to JavaScript — and the port is what proved the engine was clean.

> **Say this if asked "why four projects instead of one big one?"**
> Because each one was a real answer to a real problem, and a person can read and judge one project properly. Four small, finished, tested things are more convincing than one large, unfinished thing.

Part 03

The accounting you need

This is the part a computer science student does not have. The rules are public, they are specific, and you can be checked on them. Learn them properly and you are never the weakest person in the room.

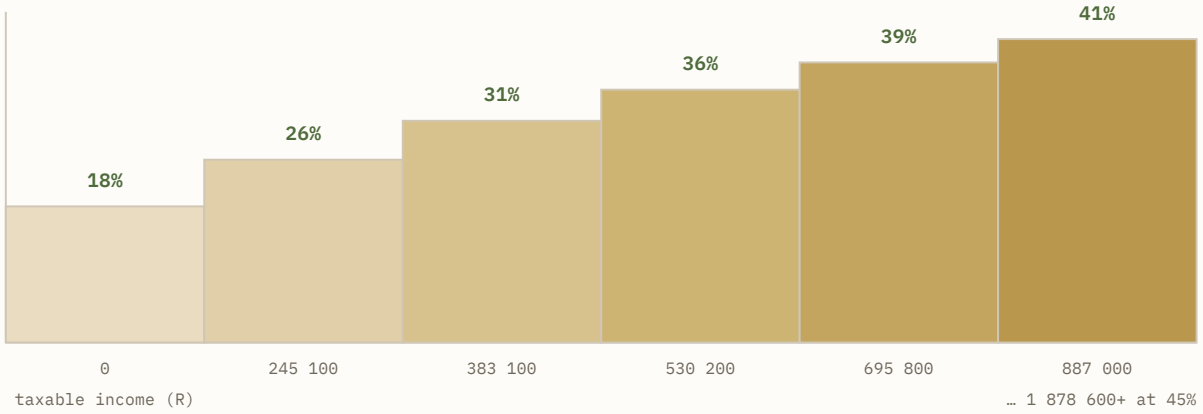
- Progressive slab rates and the annualised method
- Rebates, thresholds and the medical tax credit
- UIF, SDL and the section 11F pension cap
- VAT: output, input, periods and registration
- EMP201, EMP501, IRP5 and the deadline calendar

03 The accounting you need

The slabs: tax is progressive, not flat

South African income tax is **progressive**. Each band has its own rate, and you only pay that rate on the part of your income that falls inside it. That is why the tax table is written as "R44 118 **plus** 26% of the amount over R245 100" and not "26% of everything".

2027 TAX YEAR · 1 MAR 2026 – 28 FEB 2027



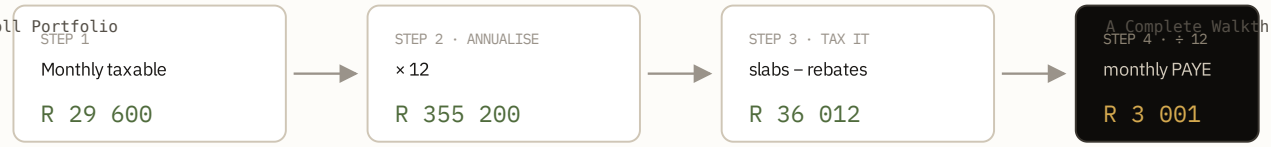
Each band taxes only the slice of income inside it. The 45% top band is not shown.

> **The worked example to have ready.** A salary of R30 000 a month:
>
> **1.** Annualised taxable income = R30 000 × 12 = **R360 000**
> **2.** It falls in the second band, so tax = R44 118 + 26% × (R360 000 − R245 100) = R44 118 + R29 874 = **R73 992**
> **3.** Less the primary rebate of **R17 820** = **R56 172** a year
> **4.** Monthly PAYE = R56 172 ÷ 12 = **R4 681**
> **5.** Plus UIF of 1% capped at R17 712 = **R177.12**
> **6.** Net pay = R30 000 − R4 681 − R177.12 = **R25 141.88**

The annualised method: the rule people get wrong

Because the brackets are built around a **year** of income, you cannot tax a month on its own. The month's taxable income is annualised, taxed, and divided by twelve again.

THE ANNUALISED METHOD



The catch worth knowing: someone who joins in November is taxed as if they would earn a full year, so the employer over-deducts for them. The year-end reconciliation in the payroll suite works out the difference and puts it right on the last payslip of the year.

The same four steps run for every payslip. The annualisation is why a December bonus does not push somebody into a higher bracket for the whole year.

The rules that catch people out

RULE	WHAT IT ACTUALLY SAYS	WHY IT MATTERS
Rebates	Primary R17 820 for everyone. Add R9 765 at 65, and a further R3 249 at 75.	Rebates are a **credit against the tax**, not an exemption from income. They are why someone on R99 000 pays nothing at all.
Medical tax credit	R376 a month for the member, R376 for the first dependant, R254 for each one after (2027).	Also a credit against the tax, and you keep the **higher** of the table amount and what you actually pay.
Section 11F	A pension deduction is capped at the lowest of the actual contribution, 27.5% of remuneration, and R350 000 a year.	Without the cap, a high earner could shelter most of their income and pay no tax at all.
UIF	1% from the employee, 1% from the employer, on remuneration up to R17 712 a month.	So the maximum anyone ever contributes is R177.12 a month, and commission earners with half their pay in commission are excluded.
SDL	1% of the leviable amount, and employers with an annual payroll below R500 000 pay none.	A business cost, not a deduction from the employee. It is the employer's own money.
VAT	15% on supplies, 15% claimed back on inputs. Mandatory registration above R2.3m a year, voluntary above R120 000 (both from 1 April 2026).	Output tax less input tax. If the input is bigger, you claim the difference back.
Interest	The first R23 800 a year is exempt (R34 500 at 65 and over).	The exemption is used up first; only what is above it is taxable.
Dividends	Exempt from income tax, but 20% is withheld at source as a final tax.	You are done with it — there is nothing more to pay on a South African dividend.

The three forms an employer files

MONTHLY **EMP201**

The monthly employer declaration. It declares the total payment and splits it between PAYE, UIF and SDL. Due by the **7th of the month** after the month it covers — and if the 7th is a Saturday, the Friday before.

YEARLY **EMP501**

The annual employer reconciliation. It proves that what was declared on the twelve EMP201s is the same as what the IRP5 certificates say was paid to the employees. Any difference has to be explained.

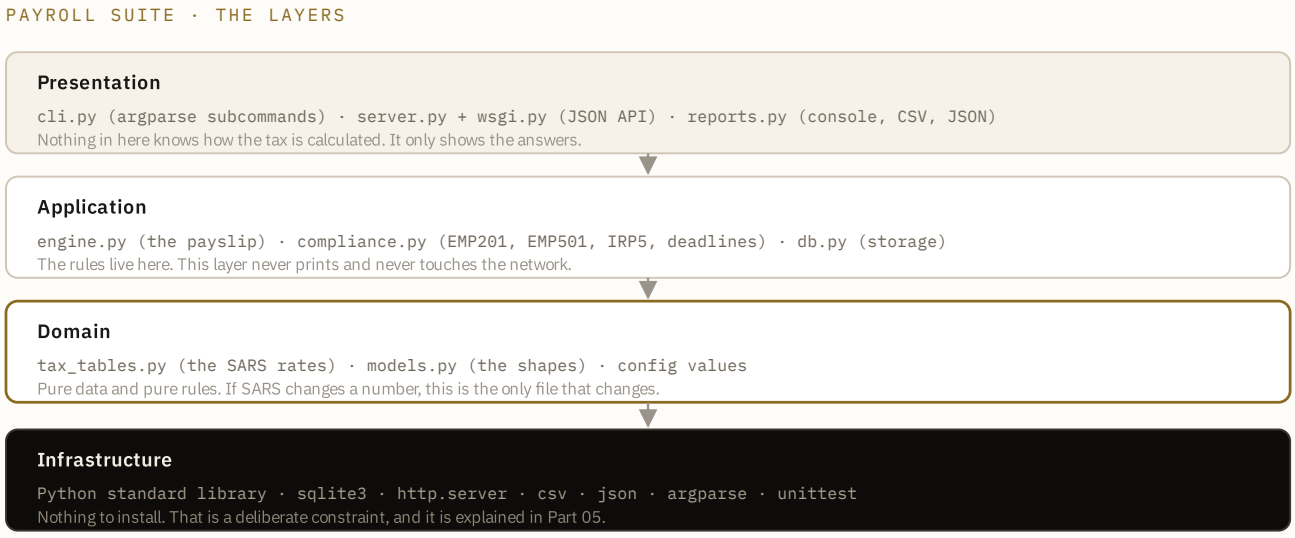
PER EMPLOYEE **IRP5**

The employee's tax certificate for the year: what they were paid, what was deducted, and what the tax came to. Issued within 60 days of the end of the tax year.

> **The best thing to say about the deadline logic:** a deadline that falls on a public holiday is not on the public holiday. My code rolls every date back to the previous business day, and there is a test that checks an October payroll is due on Friday 6 November 2026, because the 7th is a Saturday. Small detail, but it is the kind of detail that separates a real payroll from a toy one.

04 How the code fits together

The same shape runs through all three Python projects, and it is the single idea that makes them testable.

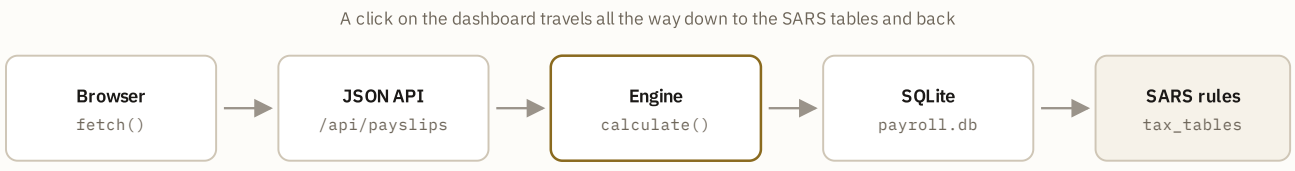


Every dependency points downwards. The tax rules do not know that a database or a terminal exists.

Why that matters

Because the engine never imports the database, you can test the whole tax calculation without one. **58 of the 148 checks in the payroll suite test the engine on its own**, with hand-worked numbers, and they run in a fraction of a second. If the engine had a database in it, every one of those tests would need a database, and you would stop writing them.

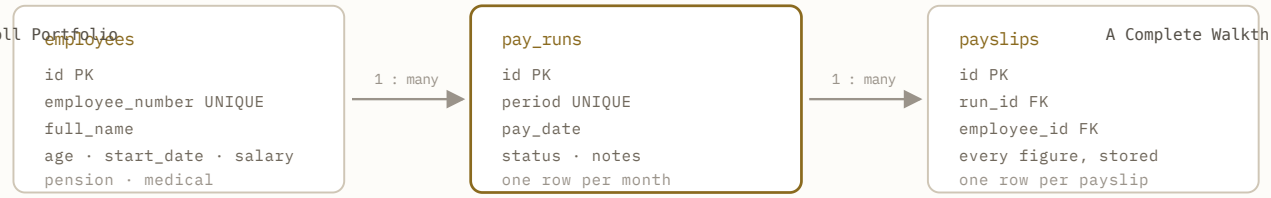
The request path, end to end



The same shape in the browser, except the last two boxes are the JavaScript engine and the tax tables it carries.

The data model

Four tables, and the relationships between them are the whole story: a pay run is a month, a payslip is an employee inside a pay run.



A `meta` table stamps the schema version, so an old database keeps working when the code moves on. Foreign keys are enforced, and there are indexes on run, employee and period.

The four tables. Nothing more complicated was needed, and nothing more complicated is easier to explain.

Part 05

The concepts, one by one

Twenty-two ideas, each with what it is, where it lives in the code, why it matters, and a sentence you can say out loud when somebody asks about it.

The accounting — 7

The employer and the forms — 4

The engineering — 7

The craft — 4

05 The concepts, one by one

Start with the seven that only a finance student would have.

A1 Progressive slab rates

WHAT IT IS — Tax is charged band by band. Each band has its own rate, and the rate applies only to the slice of income inside it. The SARS table is written as "R44 118 **plus** 26% of the amount over R245 100".

WHERE IT LIVES — `tax_tables.py → TAX_YEARS["2027"]["brackets"]`, and the arithmetic in `engine.py → annualised_tax()`.

WHY IT MATTERS — Getting this wrong means every number downstream is wrong. It is the foundation everything else sits on.

> **SAY IT LIKE THIS:**
> "South African tax is progressive, so I don't apply one rate to the whole salary — I find the band the annualised income falls in and apply that rate only to the part above the band's floor."

A2 The annualised method

WHAT IT IS — Because the brackets are built around a year of income, a month cannot be taxed alone. The month's taxable income is multiplied by twelve, taxed, and divided by twelve again.

WHERE IT LIVES — `engine.py → annualised_tax()`, and the same idea in `portfolio/assets/js/tax.js`.

WHY IT MATTERS — It smooths the tax: a December bonus does not push somebody into a higher bracket for the whole year, and a month with no overtime does not drop them into a lower one. It also creates the mid-year-joiner problem that the year-end reconciliation exists to fix.

> **SAY IT LIKE THIS:**
> "I annualise the month's taxable income, tax that, then divide by twelve to get the PAYE. It means bonuses get spread rather than hitting one hard month."

A3 Rebates and tax thresholds

WHAT IT IS A rebate is a **credit against the tax**, not an exemption from income. Primary R17 820 for everyone, plus R9 765 at 65 and R3 249 at 75. The threshold is where the rebate exactly cancels the tax — R99 000 for someone under 65 in 2027.

WHERE IT LIVES `tax_tables.py → rebates`, applied in `engine.py → annualised_tax()`.

WHY IT MATTERS It is why somebody earning R99 000 pays no tax at all, and why the secondary rebate only exists for pensioners. Treat rebates as an exemption and you will be wrong by R17 820 for most people.

> **SAY IT LIKE THIS:**
> "A rebate comes off the tax after the slab rates, not off the income before them — that difference is worth R17 820 to almost everybody."

A4 The medical scheme fees tax credit

WHAT IT IS Also a credit against the tax. R376 a month for the member, R376 for the first dependant and R254 for each dependant after that, for 2027. You keep the **higher** of the table amount and what you actually pay for your own membership.

WHERE IT LIVES `tax_tables.py → medical_credit`, and `medical_credit()` in both engines.

WHY IT MATTERS It is a real amount of money — R18 912 a year for a family — and it is the thing most people leave on the table. It also has to be applied **after** the rebates, not before.

> **SAY IT LIKE THIS:**
> "The medical credit is a rebate too, and you take the higher of the tabulated amount and what you actually pay — so a member on an expensive scheme is not penalised."

A5 Section 11F: the pension cap

WHAT IT IS A pension or provident fund contribution is deductible, but only up to the lowest of three limits: the actual contribution, 27.5% of remuneration, and R350 000 a year.

WHERE IT LIVES `payroll.py → allowance_for_retirement()` and `engine.py → calculate_pension()`.

WHY IT MATTERS Without the cap a high earner could put most of their salary into a pension and pay no tax at all. The cap is the anti-avoidance rule, and it is the kind of thing that makes the tax calculation real rather than arithmetic.

> **SAY IT LIKE THIS:**
> "The pension deduction is capped at 27.5% of pay and R350 000 a year, whichever is smaller — that cap is what stops the deduction from becoming a tax-avoidance scheme."

A6 UIF and the earnings ceiling

WHAT IT IS One percent from the employee and one percent from the employer, on remuneration, but only up to R17 712 a month. So nobody ever contributes more than R177.12 a month, whatever they earn. Commission earners whose commission is half or more of their pay are excluded entirely.

WHERE IT LIVES `uif.py` and `engine.py → calculateUIF()`.

WHY IT MATTERS It is the clearest example in the project of a rule that a naive implementation gets wrong — applying 1% to the whole salary is the obvious mistake.

> **SAY IT LIKE THIS:**
> "UIF is one percent each side but only on the first R17 712 of pay, so it caps out at R177.12 a month — and commission earners are excluded."

A7 SDL and the small-employer exemption

WHAT IT IS One percent of the leviable amount — remuneration **after** the allowable deductions — and employers with an annual payroll below R500 000 pay none of it. It is a business cost; you may not take it off the employee.

WHERE IT LIVES `payroll.py → calculate_sdl()`, with the exemption at `config.py → SDL_ANNUAL_PAYROLL_EXEMPTION`.

WHY IT MATTERS It is the difference between a small business's cost base and a large one's, and it is decided by the **employer's** annual payroll, not the employee's. Getting that scope wrong is a classic bug.

> **SAY IT LIKE THIS:**
> "SDL is a percent of the leviable amount and it's the employer's cost, and below R500 000 of annual payroll the employer is exempt entirely."

05 The employer's side

B1 Output tax, input tax, and the net

WHAT IT IS — Output tax is 15% of what you charged. Input tax is 15% of what you can claim back. The difference is what you pay SARS; if the input is bigger, you claim the difference back.

WHERE IT LIVES — `vat.py → calculate_vat()` and `vat.js → calculateVat()`.

WHY IT MATTERS — It is simpler than it looks and easier to get wrong in the other direction — a business that over-claimed input tax is committing an offence.

> **SAY IT LIKE THIS:**
> "Output tax less input tax is the net position, and a negative net is a refund rather than a payment."

B2 The six bi-monthly VAT periods

WHAT IT IS — A tax year runs 1 March to the end of February, and a vendor files a VAT201 every two months: March–April, May–June, July–August, September–October, November–December, January–February. Businesses with taxable supplies over R30 million file monthly instead.

WHERE IT LIVES — `vat.py → vat_periods()`, mirrored in `vat.js`.

WHY IT MATTERS — Each period has a **different** deadline, and the deadline is in the month **after** the period ends — so the March–April period is due in May, not in April.

> **SAY IT LIKE THIS:**
> "Six two-month periods, and the return for a period is due in the month after that period ends — the March–April one is due in May."

B3 The EMP201, and the seventh of the month

WHAT IT IS — The monthly declaration: the total payment, split between PAYE, UIF and SDL. Due within seven days after the month ends, so by the 7th of the following month.

WHERE IT LIVES — `compliance.py → emp201()` and `emp201_due_date()`.

WHY IT MATTERS — Late payment carries a 10% penalty plus daily interest, and the declaration is the employer's own liability — if you get it wrong, the money is still owed.

> **SAY IT LIKE THIS:**
> "The EMP201 declares the total and splits it three ways, and it's due by the seventh of the next month — or the Friday before, if the seventh is a weekend."

B4 The EMP501 reconciliation

WHAT IT IS Once a year, the employer proves that the twelve EMP201 declarations add up to what the IRP5 certificates say was actually paid to the employees. Any difference has to be explained.

WHERE IT LIVES `compliance.py → emp501()`, and the same check is in `tests/test_compliance.py`.

WHY IT MATTERS It is the one place where the employer's tax year closes. A payroll that produces payslips but cannot reconcile them is not finished.

> **SAY IT LIKE THIS:**
> "The EMP501 reconciles the twelve monthly declarations against the tax certificates, so the number the employer declared and the number the employee was taxed on are proven to be the same."

05 The engineering concepts

E1 The standard library only

WHAT IT IS — No pip install, no requirements.txt, no virtual environment. `sqlite3`, `http.server`, `json`, `csv`, `argparse`, `unittest` and `datetime` do all of it.

WHERE IT LIVES — Every Python file. The `pyproject.toml` lists `dependencies = []`.

WHY IT MATTERS — It means the project runs on any machine with Python, forever, with nothing to break. It is also a discipline: you learn the tool instead of the wrapper around it.

> **SAY IT LIKE THIS:**
> "I used nothing but the standard library, so the whole thing runs on any machine with Python and there is no install step and nothing to go stale."

E2 Separation of concerns

WHAT IT IS — The tax rules, the storage and the presentation are three separate layers, and they only point one way. The engine does not know a database exists; the CLI does not know how the tax is calculated.

WHERE IT LIVES — `engine.py` (rules), `db.py` (storage), `cli.py` and `server.py` (presentation).

WHY IT MATTERS — It is why 58 of the 148 checks can test the tax calculation on its own, with no database, in milliseconds. Layered code is not slower to write — it is faster to change.

> **SAY IT LIKE THIS:**
> "The engine never touches a database or the terminal, so the tax can be tested in isolation, and the same engine serves both the CLI and the API."

E3 Dataclasses as the domain model

WHAT IT IS — Every real thing in the domain — an employee, a pay run, a payslip — is a dataclass with named fields, plus a `to_row()` and `from_row()` pair so it can go into and come out of SQLite without an ORM.

WHERE IT LIVES — `payroll_suite/models.py`.

WHY IT MATTERS — The names in the code are the names in the accounting. `medical_dependants` means the same thing in the code, in the database and in the conversation with an employer.

> **SAY IT LIKE THIS:**
> "I modelled the domain as dataclasses with a row mapping, so the code reads like the accounting and there is no ORM in the way."

E4 SQLite, foreign keys and schema stamping

WHAT IT IS — A single file database. Foreign keys are switched on, the schema is created with `IF NOT EXISTS`, and a `meta` table stamps the schema version so an old database keeps working when the code moves on.

WHERE IT LIVES — `payroll_suite/db.py`.

WHY IT MATTERS — One file means the whole business's payroll can be copied onto a USB stick, backed up by copying it, and shipped to a client as one attachment.

> **SAY IT LIKE THIS:**
> "The whole payroll is one SQLite file, so backup is copying it, and the schema stamps its version so old data never breaks."

E5 The WSGI adapter

WHAT IT IS — Most free Python hosts want a WSGI callable, not a script. The suite ships `wsgi.py`, which wraps the same API in a WSGI application, so the identical code runs on a laptop and on a free host.

WHERE IT LIVES — `payroll_suite/wsgi.py`.

WHY IT MATTERS — It means "run it locally" and "host it for free" are the same code, not two implementations that drift apart.

> **SAY IT LIKE THIS:**
> "I wrote a small WSGI adapter so the same code runs on a free host as well as locally, rather than maintaining two servers."

E6 A JSON API with real error handling

WHAT IT IS — Eight endpoints, a router, a typed error class, and errors that come back as JSON with a message somebody can act on. The dashboard only ever talks to it in JSON.

WHERE IT LIVES — `payroll_suite/server.py`, and the same routes again in `wsgi.py`.

WHY IT MATTERS — The API is what makes the dashboard possible without a framework, and it means the calculations can be driven by anything that can send JSON.

> **SAY IT LIKE THIS:**
> "Eight JSON endpoints, no framework, and the errors come back as readable messages rather than stack traces."

E7 Test fixtures worked out by hand

WHAT IT IS — Every expected number in the test suite was calculated by hand from the SARS tables and typed in as a literal. No test computes its own expectation with the same code it is testing.

WHERE IT LIVES — `tests/test_engine.py`, `tests/test_vat.py`, `tests/test_payroll.py` and `tests/test_compliance.py`.

WHY IT MATTERS — It is the difference between testing and checking that a function still does what it did. A test that recomputes the answer proves nothing.

> **SAY IT LIKE THIS:**
> "Every expected figure in the tests is a literal I worked out from the SARS tables by hand, so the test is an independent check rather than a snapshot of the code."

05 The craft concepts

C1 The conceit: the site is a workbook

WHAT IT IS — Rather than a scrolling page with sections, the site is a spreadsheet: a formula bar, one open sheet, sheet tabs, a status bar, and a name box. Click any number and the formula bar shows how it was worked out.

WHERE IT LIVES — `portfolio/index.html` and `portfolio/assets/css/styles.css`, section 27.

WHY IT MATTERS — It is the thing that makes the site memorable, and it is not decoration — it is **the interface for the work**. A payroll person recognises a spreadsheet instantly, and the site's own subject matter explains its own design.

> **SAY IT LIKE THIS:**
> "I made the portfolio a workbook rather than a scrolling page, because that is the tool the work actually lives in, and it lets me show the formula behind every number."

C2 One authored moment

WHAT IT IS — One deliberate transition — the sheet changing — and nothing else that moves without a reason. The mouse-following cursor glow and the 3D card tilt were both removed.

WHERE IT LIVES — `portfolio/assets/js/main.js` and the `sheetIn` keyframe.

WHY IT MATTERS — Scattered motion reads as decoration; one moment reads as a decision. The two effects that were cut were also the most expensive things on the page while scrolling.

> **SAY IT LIKE THIS:**
> "There is exactly one animation — the sheet change. Everything else that moved was decoration, and two of those were the biggest cost while scrolling."

C3 The frame budget

WHAT IT IS At 60fps a frame is 16.7ms; on a 120Hz screen it is 8ms. Only `transform` and `opacity` skip layout and paint, so those are the only things worth animating. A blurred background costs a great deal more than it looks like.

WHERE IT LIVES The performance work in `styles.css` — the removal of the blurred aurora, the flat surfaces, and the deliberate use of `transform: scaleX()` rather than animating a width on the progress bar.

WHY IT MATTERS It is the difference between a site that feels cheap and one that feels considered, and it is measurable rather than a matter of taste.

> **SAY IT LIKE THIS:**
> "I only animate transform and opacity, because those skip layout and paint, and I took out the blurred background — it looked like nothing and cost a great deal."

C4 Themed browser surfaces

WHAT IT IS The scrollbar, the text selection, the caret, the focus ring and the numerals in the tables are all themed from the palette. The figures use tabular numerals so the columns line up.

WHERE IT LIVES `styles.css` — the scrollbar and selection rules, and `font-variant-numeric: tabular-nums` on every money column.

WHY IT MATTERS These are the parts of a page nobody drew, and they are the first thing that gives away that a page was assembled rather than built. For a finance site the numerals matter doubly: a column of Rand amounts that does not line up is a broken ledger.

> **SAY IT LIKE THIS:**
> "I themed the scrollbar, the selection and the caret, and the money uses tabular figures so the columns line up — in a spreadsheet, ragged numbers are a bug."

06 The design system

The site is a ledger, not a dashboard. That single decision decides the palette, the type and the motion — and it rules out the two things a portfolio site usually leans on, which are a violet gradient and a floating mouse glow.

Colour: a printed figure on a page

Warm ink for the ground, brass for the accent, olive for money in and terracotta for money out. Every colour clears 4.5:1 against every surface it is used on, in both themes.

Brass	Olive	Terracotta	Slate	Ink
#d4a94f the accent · 8.09:1	#8aa070 money in · 6.19:1	#c2705a money out · 4.86:1	#86a9b5 information · 7.05:1	#0d0c0a the ground · 15.18:1

> **What was thrown away and why.** The first version was cyan, violet and pink on near-black, with gradient text on the headings and frosted-glass cards. It looked like every other portfolio. The thing that changed it was reading the palette of a real financial document: the brass of a printed figure, the green of a credit, the red of a debit, and the warm off-white of the paper it is printed on.

Type: IBM Plex, and a ledger's numbers

Two families, and a rule: the sans for the words, the mono for anything that is a number. Every money column uses tabular figures, so the digits line up in a column.

Display IBM Plex Sans 700 31 pt / 3.5 rem −0.03em tracking	**The payroll suite**
Body IBM Plex Sans 400 10.2 pt / 1 rem 1.62 line height	The tax is annualised, taxed, and divided by twelve again — which is what smooths a bonus across the year.
Data IBM Plex Mono 500 tabular figures right aligned	R 1 234 567.89
Micro label IBM Plex Mono 500 12.5 pt floor	NET MONTHLY PAY

```
letterspaced caps
```

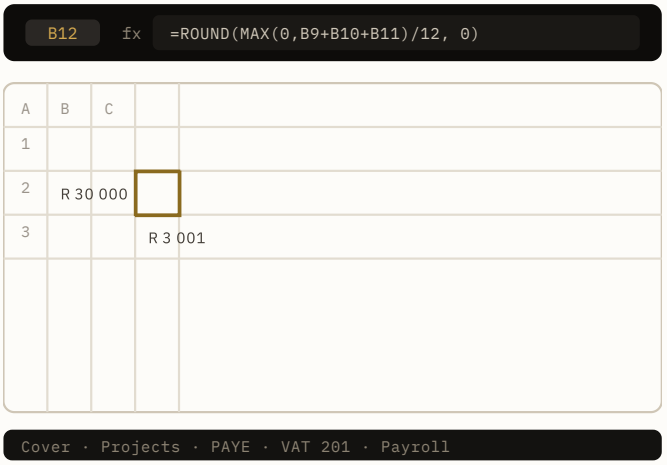

Motion: one moment, and a budget

PROPERTY	COST PER FRAME	WHAT I ANIMATE
`transform`, `opacity`	Composite only — the browser does not re-measure anything	The sheet change; the hover lifts; the progress bar
`background-color`, `color`	Paint, but no layout	Hover and focus states
`width`, `height`, `top`	Layout, then paint, then composite — a full re-measure of the page	Nothing
`filter: blur()`, `backdrop-filter`	Expensive, and the cost scales with the size of the element	Nothing — all of it was removed

> **The performance mode** in the toolbar is the honest version of this. The page measures its own frame rate while you scroll, and if more than about a third of the frames are late, it switches the decoration off and says so. It is the same idea a spreadsheet has: a degraded mode that still does the work.

Accessibility, treated as a requirement

- Every colour pair clears 4.5:1, checked in both themes
- Headings run h1 → h2 → h3 in order, with no skipped levels
- Keyboard: tabs, arrow keys between cells, `Ctrl+PageDown` between sheets, `Esc` closes the palette
- Focus rings are visible on everything, in the accent colour
- The cells are real buttons and inputs with labels
- `prefers-reduced-motion` switches every animation off
- It still works with JavaScript switched off



The workbook frame: a name box, an fx, the formula, the sheet, and the tabs.

07 Testing

262 checks across four suites, and every expected number in them was worked out by hand from the SARS tables.

114	148	31	4
checks · the calculator and the employer tool	checks · the payroll suite	checks that run in the browser console	Python versions tested on every push

What is tested, and what each suite is for

SUITE	CHECKS	WHAT IT PROVES
`test_engine.py`	58	The tax calculation itself: the slabs, the rebates, the medical credit, the s11F cap, overtime at 1.5x and 2x, the annualised method, the year-end reconciliation
`test_db.py`	27	That things survive being written and read, and that a pay run can be replaced without leaving half a month behind
`test_compliance.py`	41	The EMP201, the IRP5, the EMP501 and the deadline calendar, including a 7th that falls on a Saturday
`test_api.py`	22	That the HTTP API answers correctly, end to end, over a real socket
`test_paye.py` · `test_vat.py` · `test_payroll.py`	114	The first two projects, with the same approach

The one thing that makes the tests worth having

Every expected value is a **literal**, typed in by hand from the SARS tables:

```
# a R30 000 a month salary, 2027 tax year, 34 years old
#   annualised taxable   = 30 000 x 12                    = 355 200
#   slab tax             = 44 118 + 26% x (355 200 - 245 100) = 72 744
#   less primary rebate                                   = 17 820
#   less medical credit (R1 576 a month x 12)             = 18 912
#   annual tax           = 72 744 - 17 820 - 18 912     = 36 012
#   monthly PAYE         = 36 012 / 12                    = 3 001

close(calculatePayslip(thandi, "2027", 2_664_000).payeMonthly, 3_235)
close(calculatePayslip(thandi, "2027", 2_664_000).netMonthly, 27_087.88)
```

> **The distinction worth being able to state.** A test that recomputes the expected answer with the same function it is testing proves nothing beyond "the function is deterministic". Every expected number here is a literal I calculated myself, so a change in the code that breaks the tax shows up as a failure instead of quietly agreeing with itself.

The tests that run in the browser

The JavaScript engine is a second, independent implementation of the same rules. It carries the same hand-worked cases, and you can run them yourself in the console:

```
// open the browser console on the site and type:
Tax.selfTest()   // 12 checks — the PAYE engine
Vat.selfTest()   // 19 checks — VAT, SDL, payslips, the EMP201
```

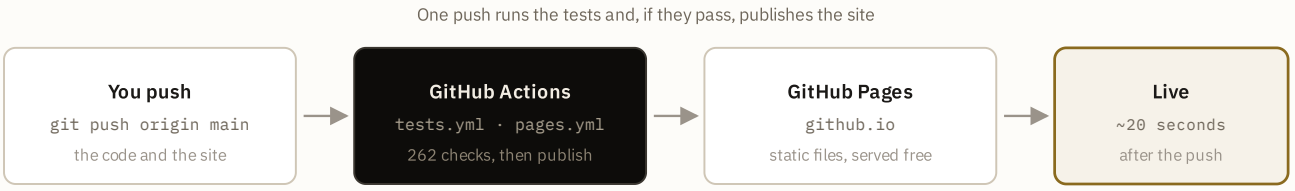
This is the thing I would point at first in an interview. Two implementations, written from the same source tables, agreeing on every figure — that is a much stronger claim than "I wrote some tests".

What is deliberately not tested

- **No mocking of the tax rules.** The whole point is that the real tables are used.
- **No test for a tax year that does not exist.** There is a validation error, but no test for the behaviour of a table from 2024, because there is no table from 2024.
- **No end-to-end browser test in CI.** The CI runs the Python suites on four versions. The browser self-tests run when you open the console, and I check them by hand before every release.
- **No load or performance testing.** Six employees is not a scale test, and I would not claim otherwise.

08 How it is hosted

The website runs on GitHub Pages, free, forever, with no server to pay for and nothing to maintain.



The site publishes itself. If the tests fail, nothing is published — which is the point of having them in CI at all.

The website — GitHub Pages

The site is pure static files, so it does not need a server at all. GitHub serves it straight from the repository, and a workflow republishes it whenever the `portfolio` folder changes. It costs nothing, there is nothing to patch, and it cannot go down.

I also added the workflow rather than using the built-in "deploy from a branch" option, because Pages can only publish the root of a branch or a `/docs` folder — and this site lives in a `portfolio` folder inside a repository that also holds the Python projects. A workflow can point anywhere, and it means the site is only rebuilt when something in it actually changed.

The payroll app — PythonAnywhere

The payroll suite needs a process that stays running and a database, so it cannot be static. It is set up to run on PythonAnywhere's free tier through the WSGI adapter.

The honest limitation: the free tier has one app, a subdomain rather than a custom domain, and on some hosts the filesystem is ephemeral — so the database would need to live somewhere that persists. For a portfolio I would rather run it from my own machine when I am demoing it than pretend it is a hosted product.

> **One thing I would say about the deployment if asked.** The site is deliberately static and dependency-free, which is not a limitation I worked around — it is the reason it still builds in five years with nobody maintaining it. The CI is the part I would defend hardest: it is what makes "push and it works" a safe thing to say.

09 The mistakes I made

This is the most useful part of the document. Anyone can list what they built. Being able to say what went wrong, and what the fix was, is what separates a person who has built something from a person who has only read about it.

1. The annualised method, done wrong

The first version divided the month's pay by the number of months the employee had worked. That produced a tax figure that **halved every month** for somebody who joined in November — because "months worked" was growing while the pay stayed the same. The numbers looked plausible and were completely wrong.

The fix was to go back to the SARS guide and read how annualisation actually works, then rebuild the model around `monthly taxable × 12`. The mid-year joiner problem is real, but it belongs to the **year-end reconciliation**, not to the monthly calculation.

> **What I would say:** "My first annualised implementation was wrong — I divided the month's pay by the months worked, which halved the tax each month for a new joiner. I caught it because the test fixtures stopped agreeing with the hand-worked numbers, and the fix was to read the SARS guide properly rather than patch the arithmetic."

2. A formula that did not add up

The formula bar shows the working for every number. To check it, I wrote a small evaluator in Python, rebuilt the formulas the JavaScript builds, and compared **what the formula evaluates to** against **what the cell displays**.

It found three bugs that no screenshot would have shown:

- the net pay formula omitted the medical contribution, so it was R1 200 out;
- the slab formula was hardcoded to the second band, so it was wrong for anyone earning more than R383 100;
- the rebate was hardcoded to the primary rebate, so it was wrong for a pensioner.

All three were invisible on the page, because the page was showing the engine's answer, which was correct. Only the *explanation* was wrong — and the explanation is the part a reader trusts.

> **What I would say:** "I wrote a spreadsheet evaluator to check that every formula in the formula bar actually evaluates to the number next to it. It found three wrong formulas — a missing medical contribution, a hardcoded tax band, and a hardcoded rebate that ignored the age rebates. The numbers on the page were right; the story about them was not."

3. A class name that meant two things

The income-breakdown bar and the toolbar at the top of the page were both called `.bar`. The toolbar's rule came later in the stylesheet, so the breakdown bar inherited a 54-pixel height and the wrong background. It rendered as an empty grey pill.

The fix was to rename the breakdown to `.breakdown` — and to notice that I only found it because I took a screenshot and looked at it. The DOM said the segment was 80 pixels wide with the right colour. It was, and it was still invisible.

> **What I would say:** "Two different components were both called '.bar', and the cascade gave the one underneath the other's styles. I found it by taking a screenshot and noticing the bar was empty while the DOM said the segments were eighty pixels wide. I renamed it, and I now give the two of them different names on purpose."

4. Reading the wrong keys

The payroll demo in the browser read `payslip.gross_remuneration` — the name the **Python** model uses. The JavaScript engine returns `grossRemuneration`. Every row in the table read "undefined".

The fix was to make the two engines use the same names, and then to look at the rendered page rather than the code.

5. Four tests that were lying

Four checks in the browser self-test were comparing a string with a numeric helper, so `Math.abs("01 Mar 2026" - "01 Mar 2026")` was `NaN` and the comparison was always false. They had never passed; they had never been run in a browser.

The fix was a comparison helper that knows about both, and then running the browser tests as part of the check before every release.

6. A page that would not lay out

I rewrote the stylesheet with a script that deleted rules by regular expression. One pattern ate a closing brace, and the page came out with the sheet area collapsed to zero height and the content invisible.

The fix was to stop patching and write the stylesheet properly, with a script that parses the file into rules, changes whole rules, and refuses to write the file if the braces do not balance. It is a small thing, but it is the reason the design pass is trustworthy.

> **The pattern behind all six:** every one of them was a case of trusting that the code was right because it looked right. The fixes were all the same shape — **make the computer check the claim**. A test that evaluates the formula. A screenshot that looks at the pixels. A brace counter that refuses to write a broken file. A test in the browser rather than only in Python.

10 Questions you might be asked

Twenty-two questions, grouped by who is likely to ask them. The answers are written to be said out loud — short, specific, and with a number in them. Read them once, then close the document and try from memory.

About the accounting

How do you calculate an employee's PAYE?Accounting

I annualise the month's taxable income, tax it on the SARS slab rates, subtract the age rebates and the medical tax credit, and divide by twelve. For a R30 000 salary that is R355 200 annualised, R44 118 plus 26% of the amount over R245 100, less the R17 820 primary rebate, less the medical credit, which gives R36 012 a year and **R3 001** a month.

Why does the tax get annualised?Accounting

Because the brackets are built around a year of income, so you cannot tax a month on its own. Annualising also smooths things: a December bonus does not push somebody into a higher band for the whole year, and a month with no overtime does not drop them into a lower one.

What is the section 11F limit and why does it exist?Accounting

A pension deduction is capped at the lowest of the actual contribution, 27.5% of remuneration, and R350 000 a year. It exists to stop high earners turning a pension fund into a tax-avoidance scheme — without it, most of a large salary could be sheltered.

A 68-year-old earns R95 000 a month. What do they pay?Accounting

Let me work it: R95 000 a month is R1 140 000 annualised, less the R4 750 pension, so R1 083 000 taxable. That is in the band over R887 000, so R259 783 plus 41%, which is R340 143. She gets the primary *and* secondary rebate because she is over 65 — R27 585 — and the medical credit. That brings it to **R24 171** a month.

What is the difference between a rebate and an exemption?Accounting

A rebate comes off the tax **after** the calculation; an exemption comes off the income **before** it. That is worth R17 820 a year to almost everybody, and the interest exemption works the other way — the first R23 800 is exempt, and only what is above it is taxed.

What is the medical tax credit, and what is the trap?Accounting

It is a credit against the tax for being on a medical scheme: R376 a month for the member, R376 for the first dependant, R254 for each one after, in 2027. The trap is that you keep the **higher** of the tabulated amount and what you actually pay, and it is a credit against the tax, not a deduction from income.

When would an employer not pay SDL?Accounting

Below R500 000 of annual payroll. It is 1% of the leviable amount — remuneration *after* the allowable deductions — and it is the employer's own cost, so it may not be taken off the employee's pay.

About the employer's compliance

What are the three forms an employer files?Compliance

The **EMP201** is monthly — the total payment split between PAYE, UIF and SDL, due by the 7th of the following month. The **IRP5** is per employee per year — their tax certificate. The **EMP501** is the annual reconciliation that proves the twelve EMP201s add up to what the IRP5s say was paid.

Why does your code roll deadlines back a day?Compliance

Because a deadline that falls on a weekend or a public holiday is not on that day — it moves to the last business day before. An October 2026 payroll is due on Friday 6 November, because the 7th is a Saturday. There is a test for exactly that case.

How does the VAT201 period work?Compliance

Six two-month periods in a tax year — March–April, May–June, and so on through to January–February — and each return is due in the month **after** the period ends. So the March–April period is due in May. Businesses over R30 million of taxable supplies file monthly instead.

What is the year-end reconciliation you built?Compliance

Because annualising taxes somebody as if they would earn a full year, a joiner is over-taxed during the year. The reconciliation works out their **real** liability on what they actually earned, compares it with what was deducted, and gives the difference back on the last payslip. In the demo, an employee who joined in November was taxed R4 180 he did not owe, and the system gives it back.

About the engineering

Why no dependencies?Engineering

It means the project runs on any machine with Python, with no install step and nothing to go stale in two years. It also keeps me honest — I had to understand the tool rather than the wrapper around it. The trade-off is that I built the small things myself: the argument parser, the HTTP server and the CSV writer.

How is the code structured?Engineering

Three layers that only point one way. The **domain** layer is the SARS rates and the shapes. The **application** layer is the engine, the compliance rules and the storage. The **presentation** layer is the CLI and the API. The engine never imports the database, which is why 58 of the 148 checks test the tax on its own.

Why is there a database at all?Engineering

Because a payroll without history is a calculator. The database holds the employees, the pay runs and the payslips, and it lets the EMP501 reconcile a whole year. It is one SQLite file, so backup is copying it and the whole business fits in an email attachment.

What does the API look like?Engineering

Eight JSON endpoints over a small WSGI application — an overview for the dashboard, the employees, the payslips for a month, the EMP201, the deadlines, the IRP5 certificates, the EMP501 reconciliation, and one endpoint that runs a payroll. No framework: the standard library's `http.server` plus about 380 lines of my own.

How do you know the tax is right?Engineering

Two ways. First, every expected figure in the test suite is a literal I calculated by hand from the SARS tables, so the tests are an independent check. Second, the tax engine exists twice — once in Python and once in JavaScript — written from the same tables, and both carry the same test cases. If the two implementations disagree, one of them is wrong, and the tests find out which.

What would you do differently?Engineering

I would write the browser tests into the CI instead of running them by hand before a release. And I would keep the tax tables in one file that both engines read, rather than maintaining the same numbers in Python and JavaScript — right now a change has to be made twice, and a test is what catches the time somebody forgets.

About the design

Why does the site look like a spreadsheet?Design

Because that is the tool the work actually lives in, and it lets me show the working. Click any number in the PAYE model and the formula bar shows the actual accounting — the slab, the rebate, the medical credit — rather than a line of code. It is a portfolio that demonstrates the portfolio.

What is the one thing you would change about the design?Design

The palette started as the usual cyan and violet on near-black, and it looked like every other portfolio. Replacing it with warm ink and brass, and taking out the frosted glass, was the change that made it feel like a document instead of a dashboard. I would do that earlier.

How do you know the site is fast?Design

Two ways. I only animate `transform` and `opacity`, because those skip layout and paint, and I removed the blurred background and the frosted glass, which were the expensive parts. And the page measures its own frame rate while you scroll — if the machine cannot keep up, it turns the decoration off and says so.

What is the performance mode for?Design

The same reason a spreadsheet has a degraded mode. The site looks the same and does exactly the same work; it just stops pretending to be a document with a glowing mouse cursor on it. I would rather degrade honestly than have the site stutter on a four-year-old laptop.

About how you work

How do you make sure you have the right rules?Process

I go to the source. The SARS rates page, the Budget tax guide, the guide for employers in respect of employees' tax. Then I put the figures in one file with the source in a comment, so there is exactly one place to change when the rules do. In 2026 the VAT registration threshold went from R1 million to R2.3 million on 1 April, and that came from the Budget guide, not from memory.

Tell me about a bug you found.Process

The annualised one, which I described in Part 9 — my first implementation halved the tax each month for a new joiner. What I would emphasise is the method: the hand-worked test figures disagreed with the code, so I went back to the SARS guide rather than adjusting the code until the test passed. That is the difference between fixing a symptom and fixing a rule.

What is next?Process

Provisional tax. The IRP6 estimator for a freelancer with no employer to deduct PAYE, the first and second payments, and the comparison against the final liability. It is the one part of the employee's side of the tax I have not built, and it is the part that most people who use the tax tables never need.

11 The numbers

For when somebody asks how big it is. All of it is in the repository, and the CI proves it still works.

4	262	11.5k	0
projects	tests, all passing	lines of code	third-party dependencies

The inventory

AREA	FILES	LINES	WHAT IS IN IT
Payroll suite (Python)	16	3 569	11 modules: the engine, the rules, the models, the database, the forms, the reports, the CLI, the API, the WSGI adapter and the seed data
Calculator & employer tool (Python)	10	1 645	Two command-line tools, the shared report formatting, the tax tables and the configuration
The portfolio site	11	4 248	The workbook: the page, the stylesheet, four scripts and the social preview image
The dashboard	3	1 009	The payroll suite's own dashboard, served by the same API
Tests	7	1 345	Four Python suites, plus the browser self-tests inside the site

What is inside the payroll suite

MODULE	LINES	JOB
`cli.py`	499	Eleven subcommands: init, add-employee, list, payrun, payslips, emp201, irp5, emp501, deadlines, export, serve
`engine.py`	408	The payslip: annualisation, overtime, the pension cap, the medical credit, UIF, SDL, and the year-end reconciliation
`server.py`	382	The JSON API and the static file server behind the dashboard
`compliance.py`	307	EMP201, IRP5, the EMP501 reconciliation and the deadline calendar
`db.py`	313	The SQLite layer: schema, migrations, and the queries
`reports.py`	236	The console reports and the CSV and JSON exports
`models.py`	200	The dataclasses and their row mappings

MODULE	LINES	JOB
`tax_tables.py`	165	Every SARS rate, in one file, with the sources
`seed.py`	131	The demo staff
`wsgi.py`	128	The WSGI adapter for free hosts

The API

ENDPOINT	RETURNS
`GET /api/overview`	The headline numbers, the trend by month, and the next deadlines
`GET /api/employees`	Everybody on the payroll
`GET /api/payslips?period=2026-10`	The payslips for a month
`GET /api/emp201?period=2026-10`	The monthly declaration and its due date
`GET /api/irp5?year=2027`	The employee tax certificates for a year
`GET /api/emp501?year=2027`	The annual reconciliation
`GET /api/deadlines?year=2027`	The employer's calendar for a year
`POST /api/run`	Runs a payroll for a month and stores it

The sources everything came from

- SARS — Rates of tax for individuals — the brackets, rebates and thresholds
- SARS — Interest and dividends — the exemptions and the 20% withholding
- SARS — Other taxes — the SDL and the VAT rates
- SARS — The EMP201 — what is declared and when
- SARS — Employer reconciliation — the EMP501
- SARS — UIF — the rates and the R17 712 ceiling
- The Budget 2026 tax guide — the VAT registration thresholds that changed on 1 April 2026

> **The disclaimer, which is also in the repository.** This is study work, not a payroll system anybody should trust with a salary. It has not had the testing, legal review and edge-case handling that a real product needs. Every figure should be checked against SARS. Keeping that on the front of the project, rather than hiding it, is part of the point.

That is the whole thing.

Four projects, 262 tests, one tax engine implemented twice, and a website that shows its working.

If somebody asks about any part of it, the answer is in Parts 3 to 5 and the number is in Part 11.